

Autor:
Ana Isabel Vegas



end2end dedication

CURSO

Persistencia Objetos con Hibernate JPA



www.ibertech.es



end2end dedication

- Tema 1** Introducción
- Tema 2** Entidades
- Tema 3** Relación entre entidades
- Tema 4** Estrategias de mapeo de herencia
- Tema 5** Interceptores - Listeners
- Tema 6** JPQL
- Tema 7** Transacciones y Concurrencia

Tema 1

Introducción

Que es JPA?

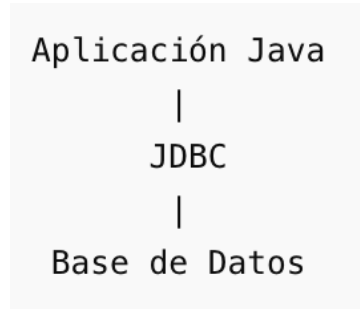
- JPA es el acrónimo de Java Persistence API y se podría considerar como el estándar de los frameworks de persistencia.
- JPA (Java Persistence API) es la especificación estándar de Java para la persistencia de datos. Define cómo las aplicaciones Java pueden almacenar, recuperar y gestionar información en bases de datos relacionales utilizando objetos Java.
- Su objetivo principal es reducir la complejidad del acceso a datos mediante una capa de abstracción que evita trabajar directamente con SQL y con las APIs JDBC de bajo nivel.

Especificacion vs Implementación

- JPA es una especificación, no una implementación. Para utilizarla se necesita un proveedor que implemente dicha especificación, como:
 - Hibernate (el más utilizado)
 - EclipseLink
 - OpenJPA

Beneficios de JPA

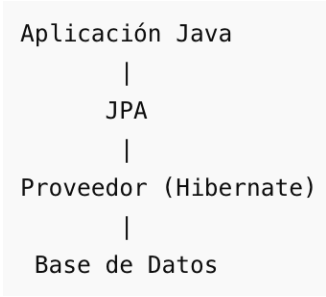
- Antes de JPA, una aplicación Java normalmente seguía este modelo:



- Los desarrolladores debían:
 - Escribir SQL manualmente.
 - Convertir filas de tablas en objetos Java.
 - Gestionar conexiones.
 - Manejar transacciones.
- Esto generaba mucho código repetitivo y difícil de mantener.

Beneficios de JPA

- JPA introduce una capa ORM (Object Relational Mapping):



- Permite establecer una correspondencia entre:

Mundo OO	Mundo Relacional
Clase	Tabla
Objeto	Fila
Atributo	Columna
Relación	Foreign Key

Arquitectura de JPA

- Elementos principales de la arquitectura:
 1. Entidades (Entities): Representan objetos del dominio que se almacenan en la base de datos.
 2. Persistence Context: Es uno de los conceptos más importantes de JPA. El Persistence Context es una especie de caché de primer nivel donde JPA mantiene las entidades gestionadas.
 3. EntityManager: Es la interfaz principal de trabajo desde donde se persisten entidades, se realizan consultas, ...etc.
 4. Proveedor JPA: La especificación necesita una implementación real, Hibernate, Open JPA,
 5. Base de datos: JPA es independiente del motor utilizado, Oracle, MySQL, ...etc.

Tema 2

Entidades

Que es una entidad?

- Consideramos una entidad al objeto que vamos a persistir o recuperar de una base de datos. Se puede ver una entidad como la representación de un registro de la tabla.
- Toda entidad ha de cumplir con los siguientes requisitos:
 - Debe implementar la interface Serializable
 - Ha de tener un constructor sin argumentos y este ha de ser público.
 - Todas las propiedades deben tener sus métodos de acceso `get()` y `set()`.
- Para crear una entidad utilizamos la anotación `@Entity`, con ella marcamos un POJO como entidad

Crear una entidad

- Para crear una entidad utilizamos la anotación `@Entity`, con ella marcamos un POJO como entidad

```
@Entity  
public class Persona implements Serializable
```

Anotaciones en JPA

- En este apartado vamos a ver las anotaciones más utilizadas para mapear cada una de las propiedades de la entidad contra la BBDD.

@Id

@IdClass

@Embeddable

@Embedded

@EmbeddedId

@Table

@Column

@SecondaryTable

@Enumerated

@Lob

@Basic

@Temporal

@Id

- Se utiliza para marcar una propiedad como clave primaria tanto si es una PK (Primary Key) simple, de un solo cambio o si forma parte de una PK compuesta, de varios campos.

```
@Entity
public class Persona implements Serializable {

    @Id
    private String nif;
```

@IdClass

- Esta anotación nos sirve para poder anotar una clave primaria compuesta.
- En una clase aparte definimos la clave primaria compuesta cumpliendo los siguientes requisitos:
 - La clase ha de implementar la interface Serializable.
 - En la clase creamos una propiedad por cada campo que forma parte de la PK.
 - Estas propiedades deben tener un método get() y set() cada una de ellas.
 - También debemos facilitar un constructor sin argumentos.
 - Debemos implementar los métodos equals() y hashCode().

@IdClass

```
public class PersonaPK implements Serializable{

    private Long telefono;
    private String nif;

    public PersonaPK() {
    }

    @Override
    public boolean equals(Object obj) { ... }

    @Override
    public int hashCode() { ... }
```

@IdClass

- En la entidad utilizamos la anotación @IdClass para especificar la clase que utilizamos como clave primaria compuesta.

```
@Entity
@IdClass(PersonaPK.class)
public class Persona implements Serializable {

    @Id
    @Column(name = "id_telefono", nullable = false)
    private Long telefono;

    @Id
    @Column(name = "id_nif", nullable = false)
    private String nif;
```

@Embeddable

- Cada una de las propiedades definida en la entidad, pasará a ser un campo en la tabla.
- Esta anotación nos permite embeber una clase.

```
@Embeddable
public class Direccion implements Serializable{

    private String calle;
    private String localidad;

    @Column(name="codigo_postal")
    private int cp;

    public Direccion() {
    }
}
```

@Embedded

- Con esta anotación hacemos que la propiedad de tipo Dirección se persista según la clase Embeddable creada anteriormente.
- Si no utilizásemos la anotación se generaría un campo direccion con el identificador del objeto.

```
@Embedded  
private Direccion direccion;
```

@EmbeddedId

- Es otra alternativa a la creación de claves primarias compuestas.
- Utilizamos esta anotación para especificar la propiedad que actúa como PK.

```
@Entity
public class Persona implements Serializable {

    @EmbeddedId
    PersonaPK personapk;
```

- La clase PersonaPK estará marcada como @Embeddable.

@Table

- Mediante esta anotación indicamos el nombre de la tabla donde se persistirán o recuperaran los datos de la entidad.
- Si no la utilizásemos coge como nombre de la tabla el nombre de la clase. Según el ejemplo sería Persona.

```
@Entity
@IdClass(PersonaPK.class)
@Table(name = "Ejemplo1_PERSONAS")
public class Persona implements Serializable {
```

@Column

- Lo mismo ocurre con las columnas, podemos especificar su nombre, tamaño, ...etc. Cada propiedad de la entidad puede utilizar una anotación @Column para establecer las propiedades de la columna en la tabla.
- Si no se utiliza, coge como nombre de columna el nombre de la propiedad.

```
@Id
@Column(name = "id_nif", nullable = false)
private String nif;
```

```
@Column(length = 1)
private char sexo;
```

@SecondaryTable

- Una entidad puede utilizar tablas secundarias para almacenar datos
- La anotación @SecondaryTable permite crear tablas secundarias. En esta anotación utilizamos dos atributos:
 - El atributo name; establecemos el nombre de la tabla.
 - El atributo pkJoinColumns; unimos la tabla primaria y la tabla secundaria.
- Para esta última operación utilizamos otra anotación @PrimaryKeyJoinColumn cuyos atributos son:
 - name; es el nombre del campo generado en la tabla secundaria
 - referencedColumnName; especificamos el nombre del campo de la tabla primaria por el cual se unen.

@SecondaryTable

```
@Entity
@IdClass(PersonaPK.class)
@Table(name = "Ejemplo1_PERSONAS")
@SecondaryTable(name = "Ejemplo1_CV",
    pkJoinColumns = {
        @PrimaryKeyJoinColumn(name = "id_telefono",
            referencedColumnName = "id_telefono"),
        @PrimaryKeyJoinColumn(name = "id_nif",
            referencedColumnName = "id_nif")
    })
public class Persona implements Serializable {

    @Lob
    @Basic(fetch = FetchType.LAZY)
    @Column(table = "Ejemplo1_CV")
    private String cv;
```

@Enumerated

- En Java utilizamos los tipos enumerados. Este tipo de datos no lo reconoce la base de datos, por lo cual, debemos utilizar esta anotación para indicar como persistir una propiedad de tipo enumerado.
- Lo podemos hacer de dos formas:
 - Almacenando su índice; EnumType.ORDINAL
 - Almacenando su valor; EnumType.STRING

```
@Enumerated(EnumType.STRING)  
private EstadoCivil estado;
```

@Lob

- Designa la propiedad de un campo como:
 - CLOB (Character Large Object); se utiliza para introducir un campo de texto muy grande. En nuestro ejemplo lo utilizamos para el curriculum de la persona.
 - BLOB (Binary Large Object); permite almacenar archivos binarios como por ejemplo una imagen.

```
@Lob
@Basic(fetch = FetchType.LAZY)
@Column(table = "Ejemplo1_CV")
private String cv;
```

@Basic

- Cuando utilizamos la anotación anterior es recomendable especificar el tipo de recuperación. Esto se denomina el tipo fetch.
- Para este fin utilizamos la anotación @Basic, mediante la cual especificamos si recuperamos automáticamente los datos o no.
 - FetchType.LAZY; Con este tipo fetch estamos indicando que al recuperar los datos de la entidad, el campo marcado de esta forma no se recupera hasta que no se solicite específicamente.
 - FetchType.EAGER; Este otro tipo indica que los datos referentes a este campo se recuperan de forma implícita.

@Temporal

- Utilizamos esta anotación para indicar como queremos persistir los objetos de tipo fecha. Puede adoptar las siguientes constantes:
 - TemporalType.DATE; almacena la fecha como día, mes y año.
 - TemporalType.TIME; se almacena la hora como horas, minutos y segundos.
 - TemporalType.TIMESTAMP; almacena fecha y hora juntas.

```
@Temporal(TemporalType.DATE)  
private Date fechaNacimiento;
```

Persistence.xml

- En este archivo vamos a definir las unidades de persistencia. Se necesita una unidad de persistencia por cada base de datos.
- También se necesitan distintas unidades de persistencia si elegimos diferentes estrategias de generación de tablas. Por ejemplo: si elegimos eliminar y crear la tabla para persistir datos y elegimos ninguna para efectuar lecturas.

Persistence.xml

- Cada unidad de persistencia ha de contener los siguientes datos:
 - **name**; nombre de la unidad de persistencia. Necesitamos un nombre para luego poder generar el objeto EntityManagerFactory del cual hablaremos más adelante.
 - **transaction-type**; especificamos el tipo de transaccionalidad elegida.
 - **provider**; necesitamos de un proveedor de persistencia. JPA actúa igual que JDBC, esto es, necesitamos una implementación del API para poder trabajar con la BBDD. En JDBC necesitábamos el driver de la BBDD y en JPA necesitamos de un proveedor de persistencia. En este ejemplo hemos optado por elegir TopLink.
 - **class**; aquí deberán especificarse todas las entidades que vamos a manejar. En nuestro ejemplo detallamos la entidad Persona.
 - **properties**; Como propiedades introducimos los datos necesarios para poder establecer una conexión a la base de datos. También como propiedad elegimos la estrategia de generación de tablas, pudiendo tomar estos valores:
 - **create-tables**; Intentará crear las tablas para cada operación.
 - **drop-and-create-tables**; Eliminará y creará las tablas de nuevo.
 - **Ninguno**; Ni elimina, ni crea tablas. Si optamos por este valor no aparece la propiedad en la unidad de persistencia.

EntityManager

- Podríamos decir que EntityManager es el objeto clave en JPA.
- El EntityManager realiza las operaciones CRUD (crear, leer, actualizar y eliminar) sobre las entidades.
- Además, el EntityManager también trata de mantener las entidades sincronizadas con la base de datos automáticamente.

EntityManagerFactory

- Para obtener el EntityManager necesitamos previamente obtener un objeto de tipo EntityManagerFactory.
- Se creará a partir del nombre de la unidad de persistencia facilitado como argumento.

EntityTransaction

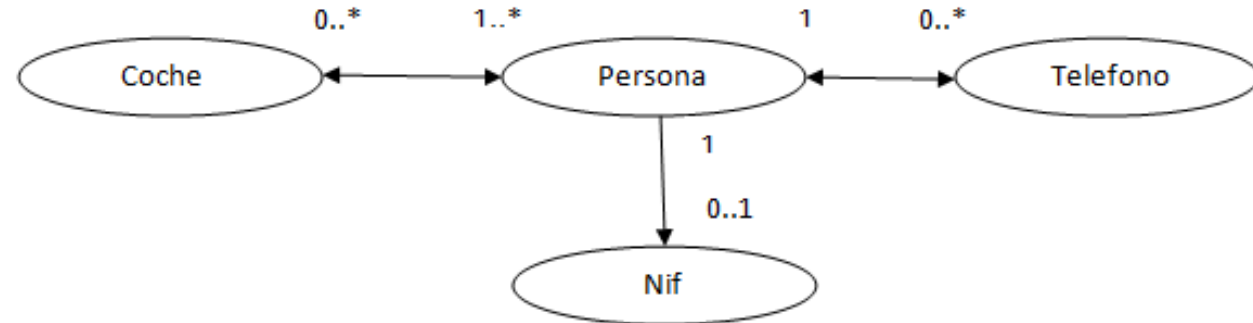
- Dependiendo de la operación que vayamos a realizar necesitamos una transacción o no. En el caso de las operaciones de persistencia es obligatorio obtener una, sin embargo en las operaciones de lectura no es necesario.
- Mediante el objeto EntityTransaction podemos obtener una transacción a partir del EntityManager.

Tema 3

Relación entre entidades

Relación entre entidades

- Para establecer las relaciones entre entidades utilizamos anotaciones como:
 - @OneToOne
 - @OneToMany
 - @ManyToOne
 - @ManyToMany



@OneToOne

- La entidad propietaria es la que consideramos más importante, en nuestro ejemplo será la entidad Persona.

```
@OneToOne(cascade=CascadeType.ALL)//cascade en entidad propietaria
@JoinColumn(name="nif_id",referencedColumnName="id")
private Nif nif;
```

- La entidad Nif actúa como subordinada. Aquí se especifica el atributo mappedBy para referenciar la propiedad nif de la entidad Persona.

```
@OneToOne(mappedBy="nif") // igual que en la entidad Persona
private Persona p;
```

@OneToMany

- La entidad que tiene la anotación @OneToMany es la entidad subordinada.

```
// quien tiene oneToMany es la entidad subordinada
@OneToMany(mappedBy="p", cascade=CascadeType.ALL)
private Set<Telefono> telefonos = new HashSet<Telefono>();

// metodos de sincronizacion
// uno por cada propiedad de tipo collection
public void addTelefono(Telefono t){
    telefonos.add(t);
    t.setP(this);
}
```

@ManyToOne

- La entidad propietaria es la entidad que contiene la anotación @ManyToOne.

```
@ManyToOne // la entidad principal es la que contiene ManyToOne
@JoinColumn(name="persona_id",referencedColumnName="id")
private Persona p;
```

@ManyToMany

- Esta anotación permite anotar relaciones muchos a muchos.
- Para poder configurar esta relación es necesario crear una tabla intermedia mediante la anotación @JoinTable. La tabla se creará con el id de persona y coche.
- También son necesarios los métodos de sincronización.

```
@ManyToMany(cascade={CascadeType.MERGE,
                    CascadeType.PERSIST,
                    CascadeType.REFRESH})
@JoinTable(name="ejemplo2_personas_coches",
            joinColumns=@JoinColumn(name="persona_id",
                                    referencedColumnName="id"),
            inverseJoinColumns=@JoinColumn(name="coche_id",
                                            referencedColumnName="id"))
private Set<Coche> coches = new HashSet<Coche>();

public void addCoche(Coche c) {
    coches.add(c);
    c.getPropietarios().add(this);
}
```

Operaciones en cascada

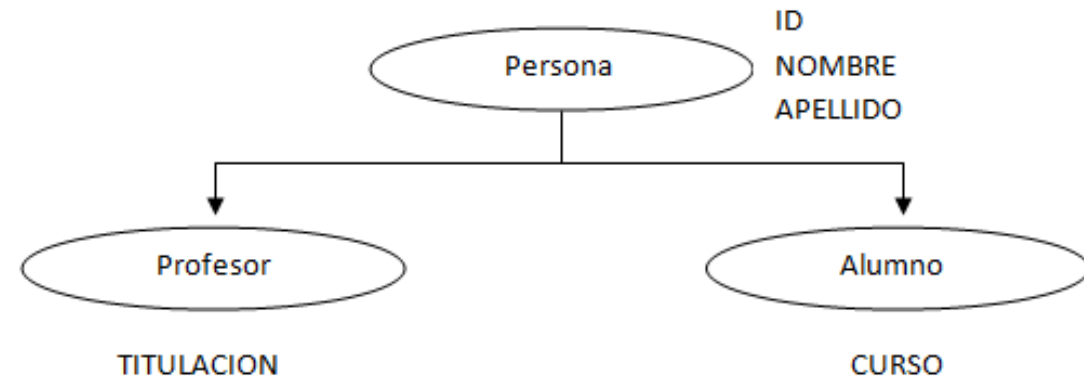
Tipo de cascada	Efecto
<code>CascadeType.MERGE</code>	Solo las operaciones <code>EntityManager.merge</code> serán propagadas a las entidades relacionadas.
<code>CascadeType.PERSIST</code>	Solo las operaciones <code>EntityManager.persist</code> serán propagadas a las entidades relacionadas.
<code>CascadeType.REFRESH</code>	Solo las operaciones <code>EntityManager.refresh</code> serán propagadas a las entidades relacionadas.
<code>CascadeType.REMOVE</code>	Solo las operaciones <code>EntityManager.remove</code> serán propagadas a las entidades relacionadas.
<code>CascadeType.ALL</code>	Todas las operaciones <code>EntityManager</code> serán propagadas a las entidades relacionadas.

Tema 4

Estrategias de mapeo de herencia

Estrategias del mapeo de herencia

- Para mapear la herencia utilizamos tres estrategias:
 - SINGLE_TABLE
 - JOINED
 - TABLE_PER_CLASS



Single_Table

- Se recogen en una sola tabla los datos de la entidad superclase y las entidades superclase.
- El campo TIPO se conoce con el nombre de discriminador y lo utilizamos para especificar si el registro es un Alumno (A) o un Profesor (P).

ID	TIPO	NOMBRE	APELLIDO	CURSO	TITULACION
3	A	Ana	Lopez	Spring	<NULL>
2	P	Jose	Sanchez	<NULL>	Ingeniero

Joined

- El modelo Joined es el más utilizado ya que se asemeja al modelo relacional utilizado en bases de datos. Sigue siendo necesario utilizar el campo discriminador.
- Las ventajas de este modelo son que no se generan valores nulos y presenta un mejor diseño.
- El inconveniente es que ahora tendremos tres tablas por lo que habrá un peor rendimiento.

ID	TIPO	NOMBRE	APELLIDO
3	A	Ana	Lopez
2	P	Jose	Sanchez

ID	CURSO
3	Spring

ID	TITULACION
2	Ingeniero

Table_Per_Class

- No todos los ORM implementan esta estrategia, por ejemplo TopLink no lo hace.
- Se genera una tabla por entidad.

ID	NOMBRE	APELLIDO
3	Ana	Lopez
2	Jose	Sanchez

ID	NOMBRE	APELLIDO	CURSO
3	Ana	Lopez	Spring

ID	NOMBRE	APELLIDO	TITULACION
2	Jose	Sanchez	Arquitecto

Tema 5

Interceptores - Listeners

Listeners de Ciclo de vida

- Un listener es un escuchador que detecta un evento. En este caso, el evento será la ejecución de un método del ciclo de vida. La siguiente tabla recoge los métodos de ciclo de vida.

Métodos de ciclo de vida	Cuando se ejecutan
@PrePersist	Antes de que el EntityManager persista la entidad
@PostPersist	Después de que la entidad sea persistida
@PostLoad	Después de que la entidad sea cargada por una query, búsqueda o operación de actualización.
@PreUpdate	Antes de que se actualice la entidad en la base de datos.
@PostUpdate	Después de que se actualice la entidad en la base de datos.
@PreRemove	Antes de que el EntityManager elimine una entidad
@PostRemove	Después de que la entidad sea eliminada

Ejemplo de Listener

```
public class MiListener {

    @PrePersist
    @PreUpdate
    public void interceptar(Persona p) {
        p.setNombre("Pepito");
        if (p instanceof Profesor) {
            System.out.println("Se va a crear un nuevo registro profesor");
        } else {
            System.out.println("Se va a crear un nuevo registro de alumno");
        }
    }
}

@Entity
@Inheritance(strategy=InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name="TIPO", discriminatorType=DiscriminatorType.STRING,
                    length=1)
@Table(name="ejemplo6_personas")
@EntityListeners(MiListener.class)
public class Persona implements Serializable {
```

Tema 6

JPQL

Qué es JPQL?

- JPQL (Java Persistence Query Language) es el lenguaje de consultas definido por JPA.
- Su sintaxis se parece a SQL, pero tiene una diferencia fundamental:
 - SQL consulta tablas y columnas.
 - JPQL consulta entidades y atributos Java.

Select

```
SELECT [DISTINCT] expression1, expression2, .... expressionN
```

```
SELECT c  
FROM Category c  
WHERE c.categoryName LIKE :categoryName  
ORDER BY c.categoryId
```

Update

```
UPDATE entityName identifierVariable
SET    single_value_path_expression1 = value1, ...
       single_value_path_expressionN = valueN
WHERE  where_clause
```

Delete

```
DELETE entityName identifierVariable  
WHERE where_clause
```

Expresiones condicionales y Operadores

Tipo de Operador	Operador
Navegación	.
Signo unario	+, -
Aritméticos	*, /, +, -
Relacionales	=, >, >=, <, <=, <>, [NOT] BETWEEN, [NOT] LIKE, [NOT] IN, IS [NOT] NULL, IS [NOT] EMPTY, [NOT] MEMBER [OF]
Lógicos	NOT, AND, OR

Funciones JPQL

Función	Descripción
CONCAT(string1, string2)	Devuelve el valor de concatenar dos literales
SUBSTRING(string, position, length)	Devuelve la subcadena comenzando en la posición y según la longitud especificada
TRIM([LEADING TRAILING BOTH] [trim_character] FROM] string_to_trimmed)	Recorta el carácter especificado a una nueva longitud. El recorte puede ser iniciales, finales o de ambos extremos.
LOWER(string)	Devuelve la cadena convertida a minúsculas
UPPER(string)	Devuelve la cadena convertida a mayúsculas
LENGTH(string)	Devuelve la longitud de la cadena
LOCATE(searchString, stringToBeSearched[initialPosition])	Devuelve la posición de una determinada cadena dentro de otra cadena. La búsqueda comienza en la posición 1 si initialPosition no se especifica.

Tema 7

Transacciones y concurrencia

Transacciones

- Una transacción se define como un conjunto de operaciones que o bien se ejecutan todas o no se ejecuta ninguna.
- Han de cumplir las propiedades ACID:
 - Atomicidad; todas las operaciones se ejecutan como un todo.
 - Consistencia; Si el sistema es consistente antes de la transacción, debe seguir siéndolo al finalizar esta.
 - Aislamiento; Nadie puede acceder a los datos involucrados en una transacción mientras esta esté en marcha.
 - Durabilidad; Los datos de la transacción tras efectuar el commit, deben ser permanentes.

EntityTransaction

```
// 1.- EntityManagerFactory
EntityManagerFactory emf =
    Persistence.createEntityManagerFactory("PU");

// 2.- EntityManager es el eje central de JPA
EntityManager em = emf.createEntityManager();

// 3.- Obtener una transaccion
EntityTransaction et = em.getTransaction();

    try{
        et.begin();

        em.persist(p1);
        em.persist(p2);
        em.persist(p3);

        et.commit();

    }catch(Exception ex){
        et.rollback();
        ex.printStackTrace();
    }finally{
        em.close();
    }
}
```

Dirty Checking

- Una de las características más potentes de JPA.

```
Cliente cliente =  
    entityManager.find(Cliente.class, 1L);  
  
cliente.setNombre("Pedro");
```

- No hacemos nada de esto:

```
UPDATE CLIENTES ...
```

```
entityManager.update(...)
```

- JPA compara el estado original con el estado actual.

```
Estado inicial  
|  
V  
Cliente(nombre="Juan")  
  
↓  
  
Cliente(nombre="Pedro")  
  
↓  
  
UPDATE generado automáticamente
```

Flush vs Commit

- Muchos desarrolladores confunden Flush con Commit.
- **Flush**
 - Sincroniza el Persistence Context con la base de datos.

```
entityManager.flush();
```

- **Commit**
 - Hace flush y confirma definitivamente.

```
tx.commit();
```

Control de concurrencia

- JPA ofrece dos estrategias:
 1. Optimistic Locking. La más utilizada.
 2. Pessimistic Locking. Menos frecuente.

Optimistic Locking

- Parte de la idea:
 - ***“Los conflictos son poco frecuentes.”***
- Se permite que varios usuarios lean simultáneamente.
- Al guardar se verifica si alguien modificó el registro.

Ejemplo con Optimistic Locking

- Todos los usuarios al leer ven la versión = 5
- Si uno de ellos la modifica, su valor ahora es 6.
- Si otro usuario intenta modificar el registro con la versión 5, Hibernate devuelve una excepción OptimisticLockException

```
@Entity
public class Cliente {

    @Id
    private Long id;

    @Version
    private Long version;

    private String nombre;
}
```

ID	NOMBRE	VERSION
1	Juan	5

Ventajas de Optimistic Locking

- Ventajas:
 - Excelente rendimiento.
 - No bloquea filas.
 - Muy escalable.
 - Ideal para aplicaciones web.
- Por eso es la estrategia recomendada por defecto.

Pessimistic Locking

- Parte de la idea:
- ***“Los conflictos son frecuentes y costosos.”***
- Se bloquea físicamente el registro.

```
Cliente cliente =  
    entityManager.find(  
        Cliente.class,  
        1L,  
        LockModeType.PESSIMISTIC_WRITE  
    );
```

- La base de datos genera algo similar a:

```
SELECT ...  
FOR UPDATE
```

- Mientras dure la transacción un usuario bloquea el registro y el resto de usuarios esperan

Pessimistic Locking

- **Ventajas:**
 - Evita conflictos completamente.
 - Útil en procesos críticos.
- **Inconvenientes:**
 - Menor rendimiento.
 - Mayor contención.
 - Riesgo de bloqueos.
- Por eso se usa mucho menos que el optimistic locking.



end2end dedication

Muchas gracias por vuestra asistencia



www.ibertech.es